

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática

NaturApp

Aplicación Móvil de Productos Naturales
con Integración Firebase

Informe Técnico — Sesión 11

Taller de Construcción de Software Móvil

Asignatura	Taller de Construcción de Software Móvil
Sesión	11 — Arquitectura Modular, Backend, Firebase
Framework	React Native + Expo SDK 52
Base de Datos	Firebase Cloud Firestore
Autenticación	Firebase Authentication
Almacenamiento	Firebase Cloud Storage
Fecha	Junio 2026

1. Introducción

NaturApp es una aplicación móvil desarrollada con React Native y Expo SDK 52, orientada a la venta de productos naturales y orgánicos. Esta versión incorpora Firebase como plataforma Backend-as-a-Service (BaaS), eliminando la necesidad de un servidor backend dedicado y proporcionando servicios de autenticación, base de datos en tiempo real y almacenamiento de archivos en la nube.

La integración de Firebase permite a la aplicación escalar de manera eficiente, reducir la complejidad de infraestructura y ofrecer una experiencia de usuario fluida con sincronización en tiempo real. El proyecto mantiene el patrón MVVM adaptado para React Native, utilizando Custom Hooks como ViewModels.

1.1. Objetivos

Implementar una arquitectura modular con Firebase como backend principal, integrando Firebase Authentication para el manejo de sesiones de usuario, Cloud Firestore como base de datos NoSQL para productos, pedidos y carritos, y Firebase Storage para el almacenamiento de imágenes. La aplicación mantiene un sistema de fallback con datos locales para garantizar su funcionamiento sin conexión a Firebase.

1.2. Alcance

La aplicación incluye: catálogo de productos con filtrado por categorías, búsqueda de productos, carrito de compras persistente por usuario, procesamiento de pedidos, perfil de usuario, y autenticación completa con registro e inicio de sesión. Toda la lógica de datos se comunica con Firebase Firestore.

2. Arquitectura del Sistema

La arquitectura sigue un enfoque modular y por capas, adaptando el patrón MVVM con Firebase como capa de servicios en la nube.

2.1. Diagrama de Arquitectura

Cliente Móvil (React Native / Expo) → Firebase SDK → Firebase Cloud Services

Capa	Descripción
Presentación (View)	Pantallas en app/ con Expo Router (file-based routing)
Lógica (ViewModel)	Custom Hooks en src/hooks/ que encapsulan estado y operaciones
Servicios	src/services/ con Firestore, Auth y Storage
Backend (Firebase)	Firebase Auth + Cloud Firestore + Cloud Storage
Datos Locales	Arrays de fallback integrados en firestoreService.js

2.2. Servicios Firebase Utilizados

Servicio	Función	SDK Módulo
Firebase Auth	Login, Registro, Sesión persistente	firebase/auth
Cloud Firestore	Base de datos NoSQL (productos, pedidos, carritos)	firebase/firestore
Cloud Storage	Almacenamiento de imágenes de productos y avatares	firebase/storage

2.3. Patrón MVVM con Firebase

En esta arquitectura, los Custom Hooks actúan como ViewModels, consumiendo los servicios de Firebase a través de la capa de servicios (firestoreService, storageService). Los hooks exponen estado reactivo (useState) y funciones de mutación que las pantallas (Views) consumen directamente. Firebase Authentication utiliza el observer onAuthStateChanged para mantener el estado de sesión sincronizado automáticamente.

3. Estructura del Proyecto

El proyecto sigue una estructura modular clara, separando presentación, lógica de negocio y servicios:

```

NaturApp_S11_Firebase/
├── app/
│   ├── _layout.js           # Pantallas (Expo Router)
│   ├── index.js             # Layout raíz (Stack)
│   ├── checkout.js          # Redirect a Home
│   ├── (tabs)/              # Pantalla de checkout
│   │   ├── _layout.js       # Tab Navigator
│   │   ├── home.js          # Layout de tabs (5 pestañas)
│   │   ├── search.js        # Catálogo de productos
│   │   ├── cart.js          # Búsqueda
│   │   ├── orders.js        # Carrito
│   │   ├── profile.js       # Historial de pedidos
│   │   └── product/[id].js  # Perfil de usuario
│   ├── auth/                # Detalle de producto
│   │   ├── login.js         # Autenticación
│   │   └── register.js     # Inicio de sesión
│   └── register.js          # Registro
├── src/
│   ├── services/            # Capa de servicios Firebase
│   │   ├── firebaseConfig.js # Inicialización Firebase
│   │   ├── firestoreService.js # CRUD Firestore + fallback
│   │   └── storageService.js  # Upload/Download de imágenes
│   ├── hooks/               # ViewModels (Custom Hooks)
│   │   ├── useAuth.js       # Autenticación Firebase
│   │   ├── useProducts.js   # Gestión de productos
│   │   ├── useCart.js       # Carrito de compras
│   │   └── useOrders.js     # Gestión de pedidos
│   └── components/          # Componentes reutilizables
│       ├── ProductCard.js   # Tarjeta de producto
│       ├── CartItemRow.js   # Fila del carrito
│       └── CategoryChips.js  # Chips de categorías
├── scripts/
│   └── seedFirestore.js     # Poblamiento de datos
├── package.json
├── app.json
├── babel.config.js
└── metro.config.js

```

4. Configuración de Firebase

4.1. Inicialización del SDK

Se utiliza el Firebase JS SDK modular (v11+) compatible con Expo. La persistencia de autenticación se configura con AsyncStorage para mantener la sesión del usuario entre cierres de la aplicación.

firebaseConfig.js

Archivo: `src/services/firebaseConfig.js`

```
// src/services/firebaseConfig.js
// =====
// CONFIGURACIÓN DE FIREBASE
// Sesión 11: Integración de Firebase
// Auth + Firestore + Storage
// =====
//
// INSTRUCCIONES DE CONFIGURACIÓN:
// 1. Ir a https://console.firebase.google.com/
// 2. Crear un nuevo proyecto o seleccionar uno existente
// 3. Agregar una app web (icono </>)
// 4. Copiar la configuración de Firebase y reemplazar los valores abajo
// 5. Habilitar Authentication > Email/Password en la consola
// 6. Crear una base de datos Firestore
// 7. Habilitar Storage
// =====

import { initializeApp } from 'firebase/app';
import { initializeAuth, getReactNativePersistence } from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import { getStorage } from 'firebase/storage';
import AsyncStorage from '@react-native-async-storage/async-storage';

// — Configuración de Firebase —
// REEMPLAZAR con tus credenciales de Firebase Console
const firebaseConfig = {
  apiKey: 'TU_API_KEY_AQUI',
  authDomain: 'tu-proyecto.firebaseio.com',
  projectId: 'tu-proyecto-id',
  storageBucket: 'tu-proyecto.appspot.com',
  messagingSenderId: '123456789',
  appId: '1:123456789:web:abcdef123456',
};

// — Inicializar Firebase —
const app = initializeApp(firebaseConfig);

// — Auth con persistencia en AsyncStorage —
// Esto mantiene la sesión del usuario entre cierres de la app
const auth = initializeAuth(app, {
  persistence: getReactNativePersistence(AsyncStorage),
});

// — Firestore (Base de datos) —
const db = getFirestore(app);

// — Storage (Almacenamiento de archivos) —
const storage = getStorage(app);

export { app, auth, db, storage };
export default app;
```

4.2. Servicio Firestore (CRUD + Fallback)

El servicio `firebaseService.js` implementa operaciones CRUD completas para productos, categorías, pedidos y carrito. Cada método incluye un bloque `try/catch` que proporciona datos locales de fallback cuando Firebase no está configurado o no hay conexión. Las colecciones de Firestore utilizadas son: `products`, `categories`, `orders`, y la subcolección `users/{userId}/cart`.

`firebaseService.js`

Archivo: `src/services/firebaseService.js`

```
// src/services/firebaseService.js
// =====
// SERVICIO FIRESTORE – CRUD de Datos
// Sesión 11: Del Módulo a la App con Firebase
// Reemplaza el backend Express con Firestore
// =====

import { db } from '../firebaseConfig';
import {
  collection, doc, getDoc, getDocs, addDoc, updateDoc, deleteDoc,
  query, where, orderBy, limit, startAfter, serverTimestamp,
} from 'firebase/firestore';

// — Catálogo local (fallback cuando Firebase no está configurado) —
const LOCAL_PRODUCTS = [
  { id: '1', name: 'Maca Negra en Polvo', description: 'Maca negra orgánica de Junín. Superalimento andino rico en aminoácidos, vitaminas y minerales. Ideal para aumentar la energía y vitalidad.', price: 45.90, category: 'superfoods', categoryName: 'Superfoods', image: 'https://images.unsplash.com/photo-1615485500704-8e990f9900f7?w=300&h=300&fit=crop', stock: 25, isActive: true, rating: 4.8, tags: ['energía', 'andino'], benefits: ['Aumenta la energía', 'Mejora la resistencia', 'Rico en hierro', 'Fortalece el sistema inmune'] },
  { id: '2', name: 'Aceite de Sacha Inchi', description: 'Aceite extra virgen de Sacha Inchi prensado en frío. Rico en Omega 3, 6 y 9. Proveniente de la Amazonía peruana.', price: 38.50, category: 'aceites', categoryName: 'Aceites', image: 'https://images.unsplash.com/photo-1474979266404-7eaabdf50494?w=300&h=300&fit=crop', stock: 18, isActive: true, rating: 4.6, tags: ['omega', 'amazónico'], benefits: ['Rico en Omega 3', 'Antiinflamatorio natural', 'Mejora la piel', 'Salud cardiovascular'] },
  { id: '3', name: 'Cápsulas de Cúrcuma', description: 'Cápsulas de cúrcuma orgánica con pimienta negra para mejor absorción. Potente antiinflamatorio y antioxidante natural.', price: 32.00, category: 'capsulas', categoryName: 'Cápsulas', image: 'https://images.unsplash.com/photo-1615485290382-441e4d049cb5?w=300&h=300&fit=crop', stock: 40, isActive: true, rating: 4.5, tags: ['antiinflamatorio'], benefits: ['Antiinflamatorio', 'Antioxidante', 'Mejora la digestión', 'Salud articular'] },
  { id: '4', name: 'Infusión de Muña', description: 'Infusión de muña andina en bolsitas filtrantes. Planta medicinal tradicional de los Andes peruanos.', price: 12.90, category: 'infusiones', categoryName: 'Infusiones', image: 'https://images.unsplash.com/photo-1564890369478-c89ca6d9cde9?w=300&h=300&fit=crop', stock: 60, isActive: true, rating: 4.3, tags: ['digestivo', 'andino'], benefits: ['Digestivo natural', 'Alivia gases', 'Refrescante', 'Sin cafeína'] },
  { id: '5', name: 'Miel de Abeja Orgánica', description: 'Miel pura de abeja de los bosques de Oxapampa. Sin aditivos ni procesamiento industrial. 100% natural.', price: 28.00, category: 'miel', categoryName: 'Miel', image: 'https://images.unsplash.com/photo-1587049352846-4a222e784d38?w=300&h=300&fit=crop', stock: 30, isActive: true, rating: 4.7, tags: ['natural', 'endulzante'], benefits: ['Energía natural', 'Antibacteriano', 'Alivia la garganta', 'Endulzante natural'] },
  { id: '6', name: 'Quinua Tricolor', description: 'Quinua tricolor orgánica: blanca, roja y negra. Grano andino con proteína completa y aminoácidos esenciales.', price: 18.50, category: 'superfoods',
```

```

categoryName: 'Superfoods', image:
'https://images.unsplash.com/photo-1586201375761-83865001e31c?w=300&h=300&fit=crop', stock: 35,
isActive: true, rating: 4.4, tags: ['proteína', 'andino'], benefits: ['Proteína completa', 'Libre de
gluten', 'Rico en fibra', 'Alto en hierro' ] },
  { id: '7', name: 'Aceite de Coco Virgen', description: 'Aceite de coco extra virgen prensado en
frío. Versátil para cocina, cuidado personal y salud.', price: 29.90, category: 'aceites',
categoryName: 'Aceites', image:
'https://images.unsplash.com/photo-1621939514649-280e2ee25f60?w=300&h=300&fit=crop', stock: 22,
isActive: true, rating: 4.5, tags: ['versátil', 'hidratante'], benefits: ['Energía rápida',
'Hidratante natural', 'Antimicrobiano', 'Versátil' ] },
  { id: '8', name: 'Cápsulas de Uña de Gato', description: 'Cápsulas de uña de gato de la Amazonía.
Propiedades inmunoestimulantes reconocidas.', price: 25.00, category: 'capsulas', categoryName:
'Cápsulas', image:
'https://images.unsplash.com/photo-1471864190281-a93a3070b6de?w=300&h=300&fit=crop', stock: 0,
isActive: true, rating: 4.2, tags: ['defensas', 'amazónico'], benefits: ['Fortalece defensas',
'Antiinflamatorio', 'Antioxidante', 'Tradición amazónica' ] },
  { id: '9', name: 'Infusión de Manzanilla y Anís', description: 'Mezcla relajante de manzanilla y
anís en bolsitas filtrantes. Ideal para tomar antes de dormir.', price: 10.50, category: 'infusiones',
categoryName: 'Infusiones', image:
'https://images.unsplash.com/photo-1544787219-7f47ccb76574?w=300&h=300&fit=crop', stock: 45, isActive:
true, rating: 4.6, tags: ['relajante', 'digestivo'], benefits: ['Relajante', 'Ayuda a dormir',
'Digestivo', 'Sin cafeína' ] },
  { id: '10', name: 'Miel con Polen', description: 'Miel de abeja enriquecida con polen de flores
silvestres. Doble beneficio nutricional en un solo producto.', price: 35.00, category: 'miel',
categoryName: 'Miel', image:
'https://images.unsplash.com/photo-1558642452-9d2a7deb7f62?w=300&h=300&fit=crop', stock: 15, isActive:
true, rating: 4.8, tags: ['nutritivo', 'energía'], benefits: ['Nutritivo', 'Energía sostenida',
'Vitaminas B', 'Antioxidante' ] },
];

const LOCAL_CATEGORIES = [
  { id: 'superfoods', name: 'Superfoods', icon: 'leaf', description: 'Superalimentos andinos y
amazónicos' },
  { id: 'aceites', name: 'Aceites', icon: 'water', description: 'Aceites naturales prensados en frío'
},
  { id: 'capsulas', name: 'Cápsulas', icon: 'medical', description: 'Suplementos en cápsulas
naturales' },
  { id: 'infusiones', name: 'Infusiones', icon: 'cafe', description: 'Infusiones y té naturales' },
  { id: 'miel', name: 'Miel', icon: 'nutrition', description: 'Mieles y derivados de abeja' },
];

// Detectar si Firebase está configurado
function isFirebaseConfigured() {
  try {
    return db && db.type === 'firestore';
  } catch {
    return false;
  }
}

// =====
// MÓDULO DE PRODUCTOS (Firestore: colección "products")
// =====
export const ProductService = {
  // Obtener todos los productos (con filtro opcional por categoría)
  getAll: async (categoryFilter = null) => {
    try {
      let q;
      if (categoryFilter) {
        q = query(
          collection(db, 'products'),
          where('isActive', '==', true),
          where('category', '==', categoryFilter),

```

```

        orderBy('name')
    );
} else {
    q = query(
        collection(db, 'products'),
        where('isActive', '==', true),
        orderBy('name')
    );
}
const snapshot = await getDocs(q);
return snapshot.docs.map(d => ({ id: d.id, ...d.data() }));
} catch (err) {
    console.log('ProductService.getAll fallback:', err.message);
    let data = LOCAL_PRODUCTS;
    if (categoryFilter) data = data.filter(p => p.category === categoryFilter);
    return data;
}
},

// Obtener un producto por ID
getById: async (id) => {
    try {
        const docRef = doc(db, 'products', id);
        const docSnap = await getDoc(docRef);
        if (!docSnap.exists()) throw new Error('Producto no encontrado');
        return { id: docSnap.id, ...docSnap.data() };
    } catch (err) {
        const product = LOCAL_PRODUCTS.find(p => p.id === id);
        if (product) return product;
        throw new Error('Producto no encontrado');
    }
},

// Buscar productos por texto
search: async (term) => {
    try {
        // Firestore no soporta búsqueda full-text nativa
        // Se obtienen todos y se filtran en el cliente
        const all = await ProductService.getAll();
        const q = term.toLowerCase();
        return all.filter(p =>
            p.name.toLowerCase().includes(q) ||
            p.description.toLowerCase().includes(q) ||
            (p.tags || []).some(t => t.toLowerCase().includes(q))
        );
    } catch (err) {
        const q = term.toLowerCase();
        return LOCAL_PRODUCTS.filter(p =>
            p.name.toLowerCase().includes(q) || p.description.toLowerCase().includes(q)
        );
    }
},

// Crear producto (admin)
create: async (productData) => {
    const docRef = await addDoc(collection(db, 'products'), {
        ...productData,
        isActive: true,
        createdAt: serverTimestamp(),
    });
    return { id: docRef.id, ...productData };
},

```



```

// Actualizar producto
update: async (id, data) => {
  const docRef = doc(db, 'products', id);
  await updateDoc(docRef, { ...data, updatedAt: serverTimestamp() });
  return { id, ...data };
},

// Eliminar producto (eliminación lógica)
delete: async (id) => {
  const docRef = doc(db, 'products', id);
  await updateDoc(docRef, { isActive: false });
},
};

// =====
// MÓDULO DE CATEGORÍAS (Firestore: colección "categories")
// =====
export const CategoryService = {
  getAll: async () => {
    try {
      const q = query(collection(db, 'categories'), orderBy('name'));
      const snapshot = await getDocs(q);
      return snapshot.docs.map(d => ({ id: d.id, ...d.data() }));
    } catch (err) {
      console.log('CategoryService.getAll fallback:', err.message);
      return LOCAL_CATEGORIES;
    }
  },
};

// =====
// MÓDULO DE PEDIDOS (Firestore: colección "orders")
// =====
const localOrders = [];

export const OrderService = {
  // Crear pedido
  create: async (userId, orderData) => {
    try {
      const docRef = await addDoc(collection(db, 'orders'), {
        userId,
        items: orderData.items,
        total: orderData.total,
        shippingAddress: orderData.shippingAddress,
        paymentMethod: orderData.paymentMethod || 'cash',
        status: 'pending',
        createdAt: serverTimestamp(),
      });
      return { id: docRef.id, ...orderData, status: 'pending', createdAt: new Date().toISOString() };
    } catch (err) {
      const order = {
        id: String(localOrders.length + 1),
        userId,
        ...orderData,
        status: 'pending',
        createdAt: new Date().toISOString(),
      };
      localOrders.unshift(order);
      return order;
    }
  },
};

// Obtener pedidos del usuario

```

```

    getByUser: async (userId) => {
      try {
        const q = query(
          collection(db, 'orders'),
          where('userId', '==', userId),
          orderBy('createdAt', 'desc')
        );
        const snapshot = await getDocs(q);
        return snapshot.docs.map(d => {
          const data = d.data();
          return {
            id: d.id,
            ...data,
            createdAt: data.createdAt?.toDate?.()?.toISOString() || data.createdAt,
          };
        });
      } catch (err) {
        return localOrders.filter(o => o.userId === userId);
      }
    },

    // Cancelar pedido
    cancel: async (orderId) => {
      try {
        const docRef = doc(db, 'orders', orderId);
        await updateDoc(docRef, { status: 'cancelled' });
        return { id: orderId, status: 'cancelled' };
      } catch (err) {
        const order = localOrders.find(o => o.id === orderId);
        if (order) order.status = 'cancelled';
        return order;
      }
    },
  };

  // =====
  // MÓDULO DE CARRITO (Firestore: subcolección "cart" dentro del usuario)
  // =====
  let localCart = { items: [] };

  export const CartService = {
    // Obtener carrito del usuario
    get: async (userId) => {
      try {
        const q = query(collection(db, 'users', userId, 'cart'));
        const snapshot = await getDocs(q);
        const items = snapshot.docs.map(d => ({ docId: d.id, ...d.data() }));
        const total = items.reduce((sum, i) => sum + i.price * i.quantity, 0);
        return { items, total, count: items.length };
      } catch (err) {
        const total = localCart.items.reduce((s, i) => s + i.price * i.quantity, 0);
        return { items: localCart.items, total, count: localCart.items.length };
      }
    },

    // Agregar al carrito
    addItem: async (userId, item) => {
      try {
        // Verificar si ya existe
        const q = query(
          collection(db, 'users', userId, 'cart'),
          where('productId', '==', item.productId)
        );

```

```

const snapshot = await getDocs(q);

if (!snapshot.empty) {
  // Actualizar cantidad
  const existingDoc = snapshot.docs[0];
  const existingData = existingDoc.data();
  await updateDoc(doc(db, 'users', userId, 'cart', existingDoc.id), {
    quantity: existingData.quantity + (item.quantity || 1),
  });
} else {
  // Agregar nuevo
  await addDoc(collection(db, 'users', userId, 'cart'), {
    productId: item.productId,
    name: item.name,
    price: item.price,
    image: item.image,
    quantity: item.quantity || 1,
  });
}
} catch (err) {
  const existing = localCart.items.find(i => i.productId === item.productId);
  if (existing) {
    existing.quantity += item.quantity || 1;
  } else {
    localCart.items.push({ ...item, quantity: item.quantity || 1 });
  }
}
},

// Actualizar cantidad
updateQuantity: async (userId, productId, quantity) => {
  try {
    const q = query(
      collection(db, 'users', userId, 'cart'),
      where('productId', '==', productId)
    );
    const snapshot = await getDocs(q);
    if (!snapshot.empty) {
      const cartDoc = snapshot.docs[0];
      if (quantity <= 0) {
        await deleteDoc(doc(db, 'users', userId, 'cart', cartDoc.id));
      } else {
        await updateDoc(doc(db, 'users', userId, 'cart', cartDoc.id), { quantity });
      }
    }
  } catch (err) {
    const item = localCart.items.find(i => i.productId === productId);
    if (item) {
      item.quantity = quantity;
      if (quantity <= 0) localCart.items = localCart.items.filter(i => i.productId !== productId);
    }
  }
},

// Eliminar del carrito
removeItem: async (userId, productId) => {
  try {
    const q = query(
      collection(db, 'users', userId, 'cart'),
      where('productId', '==', productId)
    );
    const snapshot = await getDocs(q);
    if (!snapshot.empty) {

```

```

        await deleteDoc(doc(db, 'users', userId, 'cart', snapshot.docs[0].id));
    }
} catch (err) {
    localCart.items = localCart.items.filter(i => i.productId !== productId);
}
},
// Vaciar carrito
clear: async (userId) => {
    try {
        const q = query(collection(db, 'users', userId, 'cart'));
        const snapshot = await getDocs(q);
        const deletePromises = snapshot.docs.map(d =>
            deleteDoc(doc(db, 'users', userId, 'cart', d.id))
        );
        await Promise.all(deletePromises);
    } catch (err) {
        localCart = { items: [] };
    }
},
};

```

4.3. Servicio de Storage

Firebase Storage se utiliza para almacenar imágenes de productos y avatares de usuarios. Las imágenes se convierten de URI local a Blob antes de subirse.

storageService.js

Archivo: *src/services/storageService.js*

```

// src/services/storageService.js
// =====
// SERVICIO FIREBASE STORAGE
// Sesión 11: Almacenamiento de imágenes
// Subir y obtener URLs de imágenes de productos
// =====

import { storage } from './firebaseConfig';
import { ref, uploadBytes, getDownloadURL, deleteObject } from 'firebase/storage';

const StorageService = {
    // Subir imagen de producto
    uploadProductImage: async (productId, imageUri) => {
        try {
            // Convertir URI local a blob
            const response = await fetch(imageUri);
            const blob = await response.blob();

            // Determinar extensión
            const extension = imageUri.split('.').pop()?.split('?')[0] || 'jpg';
            const fileName = `products/${productId}_${Date.now()}.${extension}`;

            // Crear referencia y subir
            const storageRef = ref(storage, fileName);
            const snapshot = await uploadBytes(storageRef, blob);

            // Obtener URL de descarga pública
            const downloadURL = await getDownloadURL(snapshot.ref);
            console.log('Imagen subida:', downloadURL);
            return downloadURL;
        }
    }
};

```

```

    } catch (error) {
      console.error('Error subiendo imagen:', error);
      throw new Error('No se pudo subir la imagen: ' + error.message);
    }
  },

  // Subir avatar de usuario
  uploadUserAvatar: async (userId, imageUri) => {
    try {
      const response = await fetch(imageUri);
      const blob = await response.blob();

      const extension = imageUri.split('.').pop()?.split('?')[0] || 'jpg';
      const fileName = `avatars/${userId}.${extension}`;

      const storageRef = ref(storage, fileName);
      const snapshot = await uploadBytes(storageRef, blob);
      const downloadURL = await getDownloadURL(snapshot.ref);

      return downloadURL;
    } catch (error) {
      console.error('Error subiendo avatar:', error);
      throw new Error('No se pudo subir el avatar: ' + error.message);
    }
  },

  // Obtener URL de descarga de una imagen
  getImageURL: async (path) => {
    try {
      const storageRef = ref(storage, path);
      return await getDownloadURL(storageRef);
    } catch (error) {
      console.error('Error obteniendo URL:', error);
      return null;
    }
  },

  // Eliminar imagen
  deleteImage: async (path) => {
    try {
      const storageRef = ref(storage, path);
      await deleteObject(storageRef);
    } catch (error) {
      console.error('Error eliminando imagen:', error);
    }
  },
};

export default StorageService;

```

5. Hooks — ViewModels

Los Custom Hooks actúan como la capa de ViewModel en la arquitectura MVVM, encapsulando toda la lógica de negocio y exponiendo estado reactivo a las pantallas.

5.1. useAuth — Autenticación Firebase

Gestiona login, registro y logout utilizando Firebase Authentication. Utiliza el observer `onAuthStateChanged` para sincronizar el estado de sesión automáticamente. Al registrar un usuario, guarda datos adicionales en Firestore (colección `users`).

useAuth.js

Archivo: `src/hooks/useAuth.js`

```
// src/hooks/useAuth.js
// =====
// Hook de Autenticación – Firebase Auth
// Sesión 11: Login, Registro, Logout con Firebase
// =====

import { useState, useEffect, useCallback } from 'react';
import { auth, db } from '../services/firebaseConfig';
import {
  onAuthStateChanged,
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  signOut,
  updateProfile,
} from 'firebase/auth';
import { doc, setDoc, getDoc } from 'firebase/firestore';

export function useAuth() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Escuchar cambios de estado de autenticación (Firebase observer)
  useEffect(() => {
    let unsubscribe;
    try {
      unsubscribe = onAuthStateChanged(auth, async (firebaseUser) => {
        if (firebaseUser) {
          // Usuario autenticado – obtener datos adicionales de Firestore
          try {
            const userDoc = await getDoc(doc(db, 'users', firebaseUser.uid));
            const userData = userDoc.exists() ? userDoc.data() : {};
            setUser({
              id: firebaseUser.uid,
              email: firebaseUser.email,
              name: firebaseUser.displayName || userData.name || 'Usuario',
              phone: userData.phone || '',
              role: userData.role || 'customer',
              photoURL: firebaseUser.photoURL || userData.photoURL || null,
            });
          } catch (err) {
            // Si Firestore falla, usar datos básicos de Auth
            setUser({
              id: firebaseUser.uid,
              email: firebaseUser.email,
              name: firebaseUser.displayName || 'Usuario',
              role: 'customer',
            });
          }
        } else {
          // Usuario no autenticado
          setUser(null);
        }
      });
    } catch (err) {
      setError(err);
    }
  }, []);

  const signIn = useCallback(async () => {
    try {
      await signInWithEmailAndPassword(auth, email, password);
    } catch (err) {
      setError(err);
    }
  }, [email, password]);

  const register = useCallback(async () => {
    try {
      await createUserWithEmailAndPassword(auth, email, password);
    } catch (err) {
      setError(err);
    }
  }, [email, password]);

  const logout = useCallback(() => {
    signOut(auth);
  }, []);

  const updateProfileData = useCallback(async (data) => {
    try {
      await updateProfile(auth.currentUser, data);
    } catch (err) {
      setError(err);
    }
  }, []);
}
```

```

    });
  }
} else {
  setUser(null);
}
setLoading(false);
});
} catch (err) {
  // Firebase no configurado - modo offline
  console.log('Firebase Auth no disponible:', err.message);
  setLoading(false);
}
return () => unsubscribe && unsubscribe();
}, []);

// Iniciar sesión con Firebase Auth
const login = useCallback(async (email, password) => {
  setLoading(true);
  setError(null);
  try {
    const result = await signInWithEmailAndPassword(auth, email, password);
    return result.user;
  } catch (err) {
    let message = 'Error de inicio de sesión';
    switch (err.code) {
      case 'auth/user-not-found':
        message = 'No existe una cuenta con este email';
        break;
      case 'auth/wrong-password':
        message = 'Contraseña incorrecta';
        break;
      case 'auth/invalid-email':
        message = 'Email inválido';
        break;
      case 'auth/too-many-requests':
        message = 'Demasiados intentos. Intenta más tarde';
        break;
      case 'auth/invalid-credential':
        message = 'Credenciales inválidas';
        break;
      default:
        // Fallback para modo desarrollo sin Firebase configurado
        if (err.message?.includes('auth/configuration-not-found') || err.code ===
'auth/api-key-not-valid') {
          setUser({
            id: 'local-1',
            email,
            name: 'Usuario Demo',
            role: 'customer',
          });
          setLoading(false);
          return { uid: 'local-1', email };
        }
        message = err.message;
      }
    }
    setError(message);
    setLoading(false);
    throw new Error(message);
  }
}, []);

// Registrar usuario nuevo con Firebase Auth + guardar perfil en Firestore
const register = useCallback(async (userData) => {

```

```

setLoading(true);
setError(null);
try {
  // Crear cuenta en Firebase Auth
  const result = await createUserWithEmailAndPassword(
    auth,
    userData.email,
    userData.password
  );

  // Actualizar displayName en Auth
  await updateProfile(result.user, { displayName: userData.name });

  // Guardar datos adicionales en Firestore
  await setDoc(doc(db, 'users', result.user.uid), {
    name: userData.name,
    email: userData.email,
    phone: userData.phone || '',
    role: 'customer',
    createdAt: new Date().toISOString(),
  });

  return result.user;
} catch (err) {
  let message = 'Error de registro';
  switch (err.code) {
    case 'auth/email-already-in-use':
      message = 'Este email ya está registrado';
      break;
    case 'auth/weak-password':
      message = 'La contraseña es muy débil (mínimo 6 caracteres)';
      break;
    case 'auth/invalid-email':
      message = 'Email inválido';
      break;
    default:
      if (err.message?.includes('configuration-not-found') || err.code ===
'auth/api-key-not-valid') {
        setUser({
          id: 'local-1',
          email: userData.email,
          name: userData.name,
          role: 'customer',
        });
        setLoading(false);
        return { uid: 'local-1', email: userData.email };
      }
      message = err.message;
    }
  }
  setError(message);
  setLoading(false);
  throw new Error(message);
}
}, []);

// Cerrar sesión
const logout = useCallback(async () => {
  try {
    await signOut(auth);
  } catch (err) {
    // Modo offline
    setUser(null);
  }
}

```



```

    }, []);

    return { user, loading, error, login, register, logout };
  }

```

5.2. useProducts — Productos

Carga productos y categorías desde Firestore. Soporta filtrado por categoría y búsqueda por texto (client-side, ya que Firestore no soporta full-text search nativo).

useProducts.js

Archivo: *src/hooks/useProducts.js*

```

// src/hooks/useProducts.js
// =====
// Hook de Productos – Firestore Service
// Sesión 11: Consulta de productos con Firebase
// =====

import { useState, useEffect, useCallback } from 'react';
import { ProductService, CategoryService } from '../services/firestoreService';

export function useProducts() {
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [selectedCategory, setSelectedCategory] = useState(null);

  // Cargar productos (opcionalmente filtrados por categoría)
  const loadProducts = useCallback(async (categoryId = null) => {
    setLoading(true);
    setError(null);
    try {
      const data = await ProductService.getAll(categoryId);
      setProducts(data);
    } catch (err) {
      console.error('Error cargando productos:', err);
      setError('No se pudieron cargar los productos');
    } finally {
      setLoading(false);
    }
  }, []);

  // Cargar categorías
  const loadCategories = useCallback(async () => {
    try {
      const data = await CategoryService.getAll();
      setCategories(data);
    } catch (err) {
      console.error('Error cargando categorías:', err);
    }
  }, []);

  // Buscar productos por texto
  const searchProducts = useCallback(async (query) => {
    setLoading(true);
    setError(null);
    try {
      const data = await ProductService.search(query);

```

```

        setProducts(data);
      } catch (err) {
        console.error('Error buscando productos:', err);
        setError('Error en la búsqueda');
      } finally {
        setLoading(false);
      }
    }, []);

    // Obtener producto por ID
    const getProductById = useCallback(async (productId) => {
      try {
        return await ProductService.getById(productId);
      } catch (err) {
        console.error('Error obteniendo producto:', err);
        return null;
      }
    }, []);

    // Filtrar por categoría
    const filterByCategory = useCallback((categoryId) => {
      setSelectedCategory(categoryId);
      loadProducts(categoryId);
    }, [loadProducts]);

    // Carga inicial
    useEffect(() => {
      loadProducts();
      loadCategories();
    }, [loadProducts, loadCategories]);

    return {
      products,
      categories,
      loading,
      error,
      selectedCategory,
      loadProducts,
      searchProducts,
      getProductById,
      filterByCategory,
    };
  }
}

```

5.3. useCart — Carrito

Maneja el carrito de compras como una subcolección de Firestore: `users/{userId}/cart`. Cada usuario tiene su propio carrito aislado. Calcula totales y cantidades de forma reactiva.

useCart.js

Archivo: `src/hooks/useCart.js`

```

// src/hooks/useCart.js
// =====
// Hook de Carrito – Firestore Subcollection
// Sesión 11: Carrito por usuario con Firebase
// =====

import { useState, useEffect, useCallback } from 'react';

```

```

import { CartService } from '../services/firestoreService';

export function useCart(userId) {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Total calculado
  const total = items.reduce((sum, item) => sum + item.price * item.quantity, 0);
  const itemCount = items.reduce((sum, item) => sum + item.quantity, 0);

  // Cargar carrito del usuario desde Firestore
  const loadCart = useCallback(async () => {
    if (!userId) return;
    setLoading(true);
    try {
      const data = await CartService.get(userId);
      setItems(data);
    } catch (err) {
      console.error('Error cargando carrito:', err);
      setError('No se pudo cargar el carrito');
    } finally {
      setLoading(false);
    }
  }, [userId]);

  // Agregar producto al carrito
  const addItem = useCallback(async (product) => {
    if (!userId) return;
    try {
      await CartService.addItem(userId, product);
      await loadCart(); // Recargar desde Firestore
    } catch (err) {
      console.error('Error agregando al carrito:', err);
      setError('No se pudo agregar el producto');
    }
  }, [userId, loadCart]);

  // Actualizar cantidad
  const updateQuantity = useCallback(async (itemId, quantity) => {
    if (!userId) return;
    try {
      if (quantity <= 0) {
        await CartService.removeItem(userId, itemId);
      } else {
        await CartService.updateQuantity(userId, itemId, quantity);
      }
      await loadCart();
    } catch (err) {
      console.error('Error actualizando cantidad:', err);
    }
  }, [userId, loadCart]);

  // Eliminar item del carrito
  const removeItem = useCallback(async (itemId) => {
    if (!userId) return;
    try {
      await CartService.removeItem(userId, itemId);
      await loadCart();
    } catch (err) {
      console.error('Error eliminando item:', err);
    }
  }, [userId, loadCart]);

```

```

// Vaciar carrito completo
const clearCart = useCallback(async () => {
  if (!userId) return;
  try {
    await CartService.clear(userId);
    setItems([]);
  } catch (err) {
    console.error('Error vaciando carrito:', err);
  }
}, [userId]);

// Cargar carrito cuando cambia el userId
useEffect(() => {
  if (userId) {
    loadCart();
  } else {
    setItems([]);
  }
}, [userId, loadCart]);

return {
  items,
  total,
  itemCount,
  loading,
  error,
  addItem,
  updateQuantity,
  removeItem,
  clearCart,
  loadCart,
};
}

```

5.4. useOrders — Pedidos

Gestiona la creación, listado y cancelación de pedidos en la colección orders de Firestore. Utiliza serverTimestamp para fechas consistentes y ordena por fecha de creación.

useOrders.js

Archivo: src/hooks/useOrders.js

```

// src/hooks/useOrders.js
// =====
// Hook de Pedidos – Firestore Service
// Sesión 11: Gestión de órdenes con Firebase
// =====

import { useState, useCallback } from 'react';
import { OrderService } from '../services/firestoreService';

export function useOrders(userId) {
  const [orders, setOrders] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  // Cargar pedidos del usuario
  const loadOrders = useCallback(async () => {

```

```

    if (!userId) return;
    setLoading(true);
    setError(null);
    try {
      const data = await OrderService.getByUser(userId);
      setOrders(data);
    } catch (err) {
      console.error('Error cargando pedidos:', err);
      setError('No se pudieron cargar los pedidos');
    } finally {
      setLoading(false);
    }
  }, [userId]);

// Crear nuevo pedido
const createOrder = useCallback(async (orderData) => {
  if (!userId) return null;
  setLoading(true);
  setError(null);
  try {
    const order = await OrderService.create({
      ...orderData,
      userId,
    });
    await loadOrders(); // Recargar lista
    return order;
  } catch (err) {
    console.error('Error creando pedido:', err);
    setError('No se pudo crear el pedido');
    return null;
  } finally {
    setLoading(false);
  }
}, [userId, loadOrders]);

// Cancelar pedido
const cancelOrder = useCallback(async (orderId) => {
  setLoading(true);
  try {
    await OrderService.cancel(orderId);
    await loadOrders();
  } catch (err) {
    console.error('Error cancelando pedido:', err);
    setError('No se pudo cancelar el pedido');
  } finally {
    setLoading(false);
  }
}, [loadOrders]);

return {
  orders,
  loading,
  error,
  loadOrders,
  createOrder,
  cancelOrder,
};
}

```

6. Componentes Reutilizables

6.1. ProductCard

Tarjeta de producto reutilizable que muestra imagen, nombre, precio, stock y botón de agregar al carrito.

ProductCard.js

Archivo: `src/components/ProductCard.js`

```
// src/components/ProductCard.js
// =====
// Componente Tarjeta de Producto
// Sesión 11: UI reutilizable
// =====

import React from 'react';
import { View, Text, Image, TouchableOpacity, StyleSheet } from 'react-native';

export default function ProductCard({ product, onPress, onAddToCart }) {
  return (
    <TouchableOpacity style={styles.card} onPress={() => onPress?.(product)} activeOpacity={0.7}>
      <Image
        source={{ uri: product.image || 'https://via.placeholder.com/150x150.png?text=Producto' }}
        style={styles.image}
        resizeMode="cover"
      />
      <View style={styles.info}>
        <Text style={styles.name} numberOfLines={2}>{product.name}</Text>
        <Text style={styles.category}>{product.category || 'General'}</Text>
        <View style={styles.row}>
          <Text style={styles.price}>$ / {(product.price || 0).toFixed(2)}</Text>
          {product.stock !== undefined && (
            <Text style={[styles.stock, product.stock <= 0 && styles.outOfStock]}>
              {product.stock > 0 ? `Stock: ${product.stock}` : 'Agotado'}
            </Text>
          )}
        </View>
        {onAddToCart && product.stock > 0 && (
          <TouchableOpacity
            style={styles.addButton}
            onPress={(e) => { e.stopPropagation?(); onAddToCart(product); }}
          >
            <Text style={styles.addButtonText}>+ Agregar</Text>
          </TouchableOpacity>
        )}
      </View>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 12,
    marginBottom: 12,
    flexDirection: 'row',
    overflow: 'hidden',
    elevation: 2,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
  },
  image: {
    width: 100,
    height: 100,
  },
  info: {
    padding: 10,
  },
  name: {
    font-size: 14,
  },
  category: {
    font-size: 12,
  },
  price: {
    font-size: 14,
  },
  stock: {
    font-size: 12,
  },
  outOfStock: {
    color: 'red',
  },
  addButton: {
    padding: 10,
  },
  addButtonText: {
    font-size: 14,
  },
});
```

```

        shadowRadius: 4,
      },
      image: {
        width: 110,
        height: 110,
        backgroundColor: '#f0f0f0',
      },
      info: {
        flex: 1,
        padding: 10,
        justifyContent: 'space-between',
      },
      name: {
        fontSize: 15,
        fontWeight: '600',
        color: '#1a1a2e',
        marginBottom: 2,
      },
      category: {
        fontSize: 12,
        color: '#888',
        marginBottom: 4,
      },
      row: {
        flexDirection: 'row',
        justifyContent: 'space-between',
        alignItems: 'center',
      },
      price: {
        fontSize: 16,
        fontWeight: '700',
        color: '#2d6a4f',
      },
      stock: {
        fontSize: 11,
        color: '#666',
      },
      outOfStock: {
        color: '#e63946',
        fontWeight: '600',
      },
      addButton: {
        backgroundColor: '#2d6a4f',
        borderRadius: 8,
        paddingVertical: 6,
        paddingHorizontal: 12,
        alignSelf: 'flex-start',
        marginTop: 4,
      },
      addButtonText: {
        color: 'fff',
        fontSize: 13,
        fontWeight: '600',
      },
    },
  });

```

6.2. CartItemRow

Fila del carrito con controles de cantidad (incrementar/decrementar) y opción de eliminar.

CartItemRow.js

Archivo: *src/components/CartItemRow.js*

```
// src/components/CartItemRow.js
// =====
// Componente Fila del Carrito
// Sesión 11: Item individual con controles de cantidad
// =====

import React from 'react';
import { View, Text, Image, TouchableOpacity, StyleSheet } from 'react-native';

export default function CartItemRow({ item, onUpdateQuantity, onRemove }) {
  return (
    <View style={styles.row}>
      <Image
        source={{ uri: item.image || 'https://via.placeholder.com/60x60.png?text=Item' }}
        style={styles.image}
      />
      <View style={styles.info}>
        <Text style={styles.name} numberOfLines={1}>{item.name}</Text>
        <Text style={styles.price}>$/ {(item.price || 0).toFixed(2)}</Text>
        <View style={styles.controls}>
          <TouchableOpacity
            style={styles.qtyButton}
            onPress={() => onUpdateQuantity?.(item.id, item.quantity - 1)}
          >
            <Text style={styles.qtyButtonText}>-</Text>
          </TouchableOpacity>
          <Text style={styles.quantity}>{item.quantity}</Text>
          <TouchableOpacity
            style={styles.qtyButton}
            onPress={() => onUpdateQuantity?.(item.id, item.quantity + 1)}
          >
            <Text style={styles.qtyButtonText}>+</Text>
          </TouchableOpacity>
          <TouchableOpacity
            style={styles.removeButton}
            onPress={() => onRemove?.(item.id)}
          >
            <Text style={styles.removeText}>Eliminar</Text>
          </TouchableOpacity>
        </View>
      </View>
      <Text style={styles.subtotal}>
        $/ {((item.price || 0) * (item.quantity || 1)).toFixed(2)}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    backgroundColor: '#fff',
    borderRadius: 10,
    padding: 10,
    marginBottom: 10,
    alignItems: 'center',
    elevation: 1,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.05,
  },
  image: {
    width: 60,
    height: 60,
    margin: 0 10,
  },
  name: {
    flex: 1,
  },
  price: {
    flex: 1,
  },
  controls: {
    flex: 1,
    display: flex,
    gap: 10,
  },
  qtyButton: {
    width: 40px,
    height: 40px,
    display: flex,
    alignContent: center,
    justify-content: center,
  },
  qtyButtonText: {
    flex: 1,
  },
  quantity: {
    flex: 1,
  },
  removeButton: {
    width: 40px,
    height: 40px,
    display: flex,
    alignContent: center,
    justify-content: center,
  },
  removeText: {
    flex: 1,
  },
  subtotal: {
    flex: 1,
  },
});
```



```

        shadowRadius: 2,
      },
      image: {
        width: 60,
        height: 60,
        borderRadius: 8,
        backgroundColor: '#f0f0f0',
      },
      info: {
        flex: 1,
        marginLeft: 10,
      },
      name: {
        fontSize: 14,
        fontWeight: '600',
        color: '#1a1a2e',
      },
      price: {
        fontSize: 13,
        color: '#666',
        marginTop: 2,
      },
      controls: {
        flexDirection: 'row',
        alignItems: 'center',
        marginTop: 6,
      },
      qtyButton: {
        backgroundColor: '#e8f5e9',
        borderRadius: 6,
        width: 28,
        height: 28,
        justifyContent: 'center',
        alignItems: 'center',
      },
      qtyButtonText: {
        fontSize: 16,
        fontWeight: '700',
        color: '#2d6a4f',
      },
      quantity: {
        fontSize: 15,
        fontWeight: '600',
        marginHorizontal: 10,
        color: '#1a1a2e',
      },
      removeButton: {
        marginLeft: 12,
      },
      removeText: {
        fontSize: 12,
        color: '#e63946',
      },
      subtotal: {
        fontSize: 15,
        fontWeight: '700',
        color: '#2d6a4f',
        marginLeft: 8,
      },
    },
  });

```

6.3. CategoryChips

Scroll horizontal de chips para filtrar productos por categoría. Incluye opción "Todos" para mostrar todos los productos.

CategoryChips.js

Archivo: `src/components/CategoryChips.js`

```
// src/components/CategoryChips.js
// =====
// Componente Chips de Categorías
// Sesión 11: Filtro horizontal de categorías
// =====

import React from 'react';
import { ScrollView, TouchableOpacity, Text, StyleSheet } from 'react-native';

export default function CategoryChips({ categories, selected, onSelect }) {
  return (
    <ScrollView
      horizontal
      showsHorizontalScrollIndicator={false}
      contentContainerStyle={styles.container}
    >
      <TouchableOpacity
        style={[styles.chip, !selected && styles.chipActive]}
        onPress={() => onSelect?.(null)}
      >
        <Text style={[styles.chipText, !selected && styles.chipTextActive]}>Todos</Text>
      </TouchableOpacity>
      {(categories || []).map((cat) => (
        <TouchableOpacity
          key={cat.id}
          style={[styles.chip, selected === cat.id && styles.chipActive]}
          onPress={() => onSelect?.(cat.id)}
        >
          <Text style={[styles.chipText, selected === cat.id && styles.chipTextActive]}>
            {cat.icon ? `${cat.icon} ` : ''}{cat.name}
          </Text>
        </TouchableOpacity>
      )))}
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: {
    paddingHorizontal: 16,
    paddingVertical: 8,
    gap: 8,
  },
  chip: {
    paddingHorizontal: 16,
    paddingVertical: 8,
    borderRadius: 20,
    backgroundColor: '#f0f0f0',
    marginRight: 8,
  },
  chipActive: {
    backgroundColor: '#2d6a4f',
  },
  chipText: {
    fontSize: 13,
  },
});
```

```
        color: '#666',  
        fontWeight: '500',  
      },  
      chipTextActive: {  
        color: '#fff',  
        fontWeight: '600',  
      },  
    },  
  });
```

7. Pantallas de la Aplicación

7.1. Home — Catálogo

home.js

Archivo: `app/(tabs)/home.js`

```
// app/(tabs)/home.js
// =====
// Pantalla de Inicio – Lista de Productos
// Sesión 11: Home con Firebase Firestore
// =====

import React, { useEffect } from 'react';
import { View, FlatList, Text, StyleSheet, ActivityIndicator, RefreshControl } from 'react-native';
import { useRouter } from 'expo-router';
import { useProducts } from '../../src/hooks/useProducts';
import { useCart } from '../../src/hooks/useCart';
import { useAuth } from '../../src/hooks/useAuth';
import ProductCard from '../../src/components/ProductCard';
import CategoryChips from '../../src/components/CategoryChips';

export default function HomeScreen() {
  const router = useRouter();
  const { user } = useAuth();
  const { products, categories, loading, error, selectedCategory, loadProducts, filterByCategory } = useProducts();
  const { addItem } = useCart(user?.id);

  const handleProductPress = (product) => {
    router.push(`/product/${product.id}`);
  };

  const handleAddToCart = async (product) => {
    if (!user) {
      router.push('/auth/login');
      return;
    }
    await addItem(product);
  };

  return (
    <View style={styles.container}>
      <CategoryChips
        categories={categories}
        selected={selectedCategory}
        onSelect={filterByCategory}
      />

      {error && <Text style={styles.error}>{error}</Text>}

      <FlatList
        data={products}
        keyExtractor={({ item }) => item.id}
        renderItem={({ item }) => (
          <ProductCard
            product={item}
            onPress={handleProductPress}
            onAddToCart={handleAddToCart}
          />
        )}
      />
    </View>
  );
}
```

```

    refreshControl={
      <RefreshControl
        refreshing={loading}
        onRefresh={() => loadProducts(selectedCategory)}
        colors={['#2d6a4f']}
      />
    }
    ListEmptyComponent={
      !loading && (
        <View style={styles.empty}>
          <Text style={styles.emptyIcon}><img alt="leaf icon" data-bbox="458 218 473 233"/></Text>
          <Text style={styles.emptyText}>No hay productos disponibles</Text>
        </View>
      )
    }
  />
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  list: { padding: 16 },
  error: {
    color: '#e63946',
    textAlign: 'center',
    padding: 10,
    backgroundColor: '#fdecea',
    marginHorizontal: 16,
    borderRadius: 8,
  },
  empty: { alignItems: 'center', marginTop: 60 },
  emptyIcon: { fontSize: 48, marginBottom: 12 },
  emptyText: { fontSize: 16, color: '#888' },
});

```

7.2. Search — Búsqueda

search.js

Archivo: `app/(tabs)/search.js`

```

// app/(tabs)/search.js
// =====
// Pantalla de Búsqueda
// Sesión 11: Búsqueda client-side con Firestore
// =====

import React, { useState } from 'react';
import { View, TextInput, FlatList, Text, StyleSheet, TouchableOpacity } from 'react-native';
import { useRouter } from 'expo-router';
import { useProducts } from '../../src/hooks/useProducts';
import { useCart } from '../../src/hooks/useCart';
import { useAuth } from '../../src/hooks/useAuth';
import ProductCard from '../../src/components/ProductCard';

export default function SearchScreen() {
  const router = useRouter();
  const { user } = useAuth();
  const { products, loading, searchProducts, loadProducts } = useProducts();
  const { addItem } = useCart(user?.id);
  const [query, setQuery] = useState('');

```

```

const handleSearch = () => {
  if (query.trim()) {
    searchProducts(query.trim());
  } else {
    loadProducts();
  }
};

const handleAddToCart = async (product) => {
  if (!user) {
    router.push('/auth/login');
    return;
  }
  await addItem(product);
};

return (
  <View style={styles.container}>
    <View style={styles.searchRow}>
      <TextInput
        style={styles.input}
        placeholder="Buscar productos naturales..."
        value={query}
        onChangeText={setQuery}
        onSubmitEditing={handleSearch}
        returnKeyType="search"
      />
      <TouchableOpacity style={styles.searchButton} onPress={handleSearch}>
        <Text style={styles.searchButtonText}><img alt="search icon" data-bbox="478 462 492 476"/></Text>
      </TouchableOpacity>
    </View>

    <FlatList
      data={products}
      keyExtractor={({item}) => item.id}
      renderItem={({ item }) => (
        <ProductCard
          product={item}
          onPress={(p) => router.push(`/product/${p.id}`)}
          onAddToCart={handleAddToCart}
        />
      )}
      contentContainerStyle={styles.list}
      ListEmptyComponent={
        !loading && (
          <View style={styles.empty}>
            <Text style={styles.emptyIcon}><img alt="empty icon" data-bbox="458 692 472 706"/></Text>
            <Text style={styles.emptyText}>
              {query ? 'No se encontraron resultados' : 'Escribe para buscar productos'}
            </Text>
          </View>
        )
      }
    />
  </View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  searchRow: {
    flexDirection: 'row',

```

```

padding: 16,
gap: 10,
},
input: {
  flex: 1,
  backgroundColor: '#fff',
  borderRadius: 12,
  paddingHorizontal: 16,
  paddingVertical: 12,
  fontSize: 15,
  elevation: 2,
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 1 },
  shadowOpacity: 0.1,
  shadowRadius: 3,
},
searchButton: {
  backgroundColor: '#2d6a4f',
  borderRadius: 12,
  width: 48,
  justifyContent: 'center',
  alignItems: 'center',
},
searchButtonText: { fontSize: 20 },
list: { padding: 16, paddingTop: 0 },
empty: { alignItems: 'center', marginTop: 60 },
emptyIcon: { fontSize: 48, marginBottom: 12 },
emptyText: { fontSize: 16, color: '#888', textAlign: 'center' },
});

```

7.3. Cart — Carrito

cart.js

Archivo: `app/(tabs)/cart.js`

```

// app/(tabs)/cart.js
// =====
// Pantalla de Carrito
// Sesión 11: Carrito con Firestore subcollection
// =====

import React from 'react';
import { View, FlatList, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';
import { useCart } from '../../src/hooks/useCart';
import CartItemRow from '../../src/components/CartItemRow';

export default function CartScreen() {
  const router = useRouter();
  const { user } = useAuth();
  const { items, total, itemCount, loading, updateQuantity, removeItem, clearCart } =
    useCart(user?.id);

  if (!user) {
    return (
      <View style={styles.center}>
        <Text style={styles.emptyIcon}>🛒</Text>
        <Text style={styles.emptyText}>Inicia sesión para ver tu carrito</Text>
        <TouchableOpacity style={styles.loginButton} onPress={() => router.push('/auth/login')}>
          <Text style={styles.loginButtonText}>Iniciar Sesión</Text>
        </TouchableOpacity>
      </View>
    );
  }
}

```

```

        </TouchableOpacity>
      </View>
    );
  }

  if (items.length === 0) {
    return (
      <View style={styles.center}>
        <Text style={styles.emptyIcon}><img alt="Shopping cart icon" data-bbox="415 195 430 205"/></Text>
        <Text style={styles.emptyText}>Tu carrito está vacío</Text>
        <TouchableOpacity style={styles.loginButton} onPress={() => router.push('/(tabs)/home')}>
          <Text style={styles.loginButtonText}>Explorar Productos</Text>
        </TouchableOpacity>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <FlatList
        data={items}
        keyExtractor={({item}) => item.id}
        renderItem={({item}) => (
          <CartItemRow
            item={item}
            onUpdateQuantity={updateQuantity}
            onRemove={removeItem}
          />
        )}
        contentContainerStyle={styles.list}
      />

      <View style={styles.footer}>
        <View style={styles.summary}>
          <Text style={styles.summaryLabel}>Total ({itemCount} items)</Text>
          <Text style={styles.summaryTotal}>$/ {total.toFixed(2)}</Text>
        </View>
        <View style={styles.footerButtons}>
          <TouchableOpacity style={styles.clearButton} onPress={clearCart}>
            <Text style={styles.clearButtonText}>Vaciar</Text>
          </TouchableOpacity>
          <TouchableOpacity
            style={styles.checkoutButton}
            onPress={() => router.push('/checkout')}
          >
            <Text style={styles.checkoutButtonText}>Proceder al Pago</Text>
          </TouchableOpacity>
        </View>
      </View>
    );
  }
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20 },
  list: { padding: 16 },
  emptyIcon: { fontSize: 60, marginBottom: 16 },
  emptyText: { fontSize: 17, color: '#888', marginBottom: 20 },
  loginButton: {
    backgroundColor: '#2d6a4f',
    borderRadius: 10,
    paddingVertical: 12,
  }
});

```



```

    paddingHorizontal: 24,
  },
  loginButtonText: { color: '#fff', fontSize: 15, fontWeight: '600' },
  footer: {
    backgroundColor: '#fff',
    padding: 16,
    borderTopWidth: 1,
    borderTopColor: '#eee',
  },
  summary: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 12,
  },
  summaryLabel: { fontSize: 15, color: '#666' },
  summaryTotal: { fontSize: 20, fontWeight: '700', color: '#2d6a4f' },
  footerButtons: { flexDirection: 'row', gap: 10 },
  clearButton: {
    flex: 1,
    borderWidth: 1,
    borderColor: '#e63946',
    borderRadius: 10,
    paddingVertical: 12,
    alignItems: 'center',
  },
  clearButtonText: { color: '#e63946', fontWeight: '600' },
  checkoutButton: {
    flex: 2,
    backgroundColor: '#2d6a4f',
    borderRadius: 10,
    paddingVertical: 12,
    alignItems: 'center',
  },
  checkoutButtonText: { color: '#fff', fontSize: 15, fontWeight: '700' },
});

```

7.4. Orders — Pedidos

orders.js

Archivo: `app/(tabs)/orders.js`

```

// app/(tabs)/orders.js
// =====
// Pantalla de Pedidos
// Sesión 11: Historial de órdenes con Firestore
// =====

import React, { useEffect } from 'react';
import { View, FlatList, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';
import { useOrders } from '../../src/hooks/useOrders';

function OrderCard({ order, onCancel }) {
  const statusColors = {
    pending: '#f4a261',
    confirmed: '#2a9d8f',
    shipped: '#264653',
    delivered: '#2d6a4f',
    cancelled: '#e63946',
  };
};

```

```

const statusLabels = {
  pending: 'Pendiente',
  confirmed: 'Confirmado',
  shipped: 'Enviado',
  delivered: 'Entregado',
  cancelled: 'Cancelado',
};

const date = order.createdAt?.toDate
  ? order.createdAt.toDate().toLocaleDateString('es-PE')
  : new Date(order.createdAt).toLocaleDateString('es-PE');

return (
  <View style={styles.orderCard}>
    <View style={styles.orderHeader}>
      <Text style={styles.orderId}>Pedido #{(order.id || '').slice(-6).toUpperCase()}</Text>
      <View style={[styles.badge, { backgroundColor: statusColors[order.status] || '#999' }]}>
        <Text style={styles.badgeText}>{statusLabels[order.status] || order.status}</Text>
      </View>
    </View>
    <Text style={styles.orderDate}>{date}</Text>
    <Text style={styles.orderItems}>
      {(order.items || []).map((i) => `${i.name} x${i.quantity}`).join(', ')}
    </Text>
    <View style={styles.orderFooter}>
      <Text style={styles.orderTotal}>S/ ${(order.total || 0).toFixed(2)}</Text>
      {order.status === 'pending' && (
        <TouchableOpacity onPress={() => onCancel?.(order.id)}>
          <Text style={styles.cancelText}>Cancelar</Text>
        </TouchableOpacity>
      )}
    </View>
  </View>
);
}

export default function OrdersScreen() {
  const router = useRouter();
  const { user } = useAuth();
  const { orders, loading, loadOrders, cancelOrder } = useOrders(user?.id);

  useEffect(() => {
    if (user) loadOrders();
  }, [user, loadOrders]);

  if (!user) {
    return (
      <View style={styles.center}>
        <Text style={styles.emptyIcon}>🔒</Text>
        <Text style={styles.emptyText}>Inicia sesión para ver tus pedidos</Text>
        <TouchableOpacity style={styles.loginButton} onPress={() => router.push('/auth/login')}>
          <Text style={styles.loginButtonText}>Iniciar Sesión</Text>
        </TouchableOpacity>
      </View>
    );
  }

  return (
    <View style={styles.container}>
      <FlatList
        data={orders}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => <OrderCard order={item} onCancel={cancelOrder} />
      </FlatList>
    </View>
  );
}

```

```

        contentContainerStyle={styles.list}
        onRefresh={loadOrders}
        refreshing={loading}
        ListEmptyComponent={
          !loading && (
            <View style={styles.center}>
              <Text style={styles.emptyIcon}><img alt="empty icon" data-bbox="458 168 475 182"/></Text>
              <Text style={styles.emptyText}>Aún no tienes pedidos</Text>
            </View>
          )
        }
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20 },
  list: { padding: 16 },
  emptyIcon: { fontSize: 60, marginBottom: 16 },
  emptyText: { fontSize: 17, color: '#888', marginBottom: 20 },
  loginButton: {
    backgroundColor: '#2d6a4f',
    borderRadius: 10,
    paddingVertical: 12,
    paddingHorizontal: 24,
  },
  loginButtonText: { color: 'fff', fontSize: 15, fontWeight: '600' },
  orderCard: {
    backgroundColor: 'fff',
    borderRadius: 12,
    padding: 14,
    marginBottom: 12,
    elevation: 2,
    shadowColor: '000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
    shadowRadius: 3,
  },
  orderHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  orderId: { fontSize: 15, fontWeight: '700', color: '#1a1a2e' },
  badge: { borderRadius: 12, paddingHorizontal: 10, paddingVertical: 4 },
  badgeText: { color: 'fff', fontSize: 11, fontWeight: '600' },
  orderDate: { fontSize: 12, color: '#888', marginTop: 4 },
  orderItems: { fontSize: 13, color: '#555', marginTop: 6 },
  orderFooter: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginTop: 10,
    borderTopWidth: 1,
    borderTopColor: '#f0f0f0',
    paddingTop: 8,
  },
  orderTotal: { fontSize: 17, fontWeight: '700', color: '#2d6a4f' },
  cancelText: { color: '#e63946', fontSize: 13, fontWeight: '600' },
});

```

7.5. Profile — Perfil

profile.js

Archivo: `app/(tabs)/profile.js`

```
// app/(tabs)/profile.js
// =====
// Pantalla de Perfil
// Sesión 11: Perfil de usuario con Firebase Auth
// =====

import React from 'react';
import { View, Text, TouchableOpacity, Image, StyleSheet, Alert, ScrollView } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';

export default function ProfileScreen() {
  const router = useRouter();
  const { user, logout } = useAuth();

  if (!user) {
    return (
      <View style={styles.center}>
        <Text style={styles.emptyIcon}>👤</Text>
        <Text style={styles.emptyText}>Inicia sesión para ver tu perfil</Text>
        <TouchableOpacity style={styles.loginButton} onPress={() => router.push('/auth/login')}>
          <Text style={styles.loginButtonText}>Iniciar Sesión</Text>
        </TouchableOpacity>
      </View>
    );
  }

  const handleLogout = () => {
    Alert.alert('Cerrar Sesión', '¿Estás seguro?', [
      { text: 'Cancelar', style: 'cancel' },
      { text: 'Salir', style: 'destructive', onPress: logout },
    ]);
  };

  return (
    <ScrollView style={styles.container} contentContainerStyle={styles.content}>
      <View style={styles.header}>
        <View style={styles.avatar}>
          {user.photoURL ? (
            <Image source={{ uri: user.photoURL }} style={styles.avatarImage} />
          ) : (
            <Text style={styles.avatarText}>{(user.name || 'U')[0].toUpperCase()}</Text>
          )}
        </View>
        <Text style={styles.name}>{user.name}</Text>
        <Text style={styles.email}>{user.email}</Text>
      </View>

      <View style={styles.section}>
        <Text style={styles.sectionTitle}>Cuenta</Text>
        <View style={styles.infoRow}>
          <Text style={styles.infoLabel}>Nombre</Text>
          <Text style={styles.infoValue}>{user.name}</Text>
        </View>
        <View style={styles.infoRow}>
          <Text style={styles.infoLabel}>Email</Text>

```

```

        <Text style={styles.infoValue}>{user.email}</Text>
      </View>
      <View style={styles.infoRow}>
        <Text style={styles.infoLabel}>Teléfono</Text>
        <Text style={styles.infoValue}>{user.phone || 'No registrado'}</Text>
      </View>
      <View style={styles.infoRow}>
        <Text style={styles.infoLabel}>Rol</Text>
        <Text style={styles.infoValue}>{user.role === 'admin' ? 'Administrador' : 'Cliente'}</Text>
      </View>
    </View>

    <View style={styles.section}>
      <Text style={styles.sectionTitle}>Firebase</Text>
      <View style={styles.infoRow}>
        <Text style={styles.infoLabel}>UID</Text>
        <Text style={styles.infoValue} numberOfLines={1}>{user.id}</Text>
      </View>
      <View style={styles.infoRow}>
        <Text style={styles.infoLabel}>Auth Provider</Text>
        <Text style={styles.infoValue}>Email / Password</Text>
      </View>
    </View>

    <TouchableOpacity style={styles.logoutButton} onPress={handleLogout}>
      <Text style={styles.logoutText}>Cerrar Sesión</Text>
    </TouchableOpacity>
  </ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  content: { padding: 16 },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20 },
  emptyIcon: { fontSize: 60, marginBottom: 16 },
  emptyText: { fontSize: 17, color: '#888', marginBottom: 20 },
  loginButton: {
    backgroundColor: '#2d6a4f',
    borderRadius: 10,
    paddingVertical: 12,
    paddingHorizontal: 24,
  },
  loginButtonText: { color: 'fff', fontSize: 15, fontWeight: '600' },
  header: {
    alignItems: 'center',
    backgroundColor: 'fff',
    borderRadius: 16,
    padding: 24,
    marginBottom: 16,
    elevation: 2,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
  },
  avatar: {
    width: 80,
    height: 80,
    borderRadius: 40,
    backgroundColor: '#2d6a4f',
    justifyContent: 'center',
    alignItems: 'center',
  },
});

```

```

    marginBottom: 12,
  },
  avatarImage: { width: 80, height: 80, borderRadius: 40 },
  avatarText: { fontSize: 32, fontWeight: '700', color: 'fff' },
  name: { fontSize: 20, fontWeight: '700', color: '1a1a2e' },
  email: { fontSize: 14, color: '888', marginTop: 4 },
  section: {
    backgroundColor: 'fff',
    borderRadius: 12,
    padding: 16,
    marginBottom: 16,
    elevation: 1,
    shadowColor: '000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.05,
    shadowRadius: 2,
  },
  sectionTitle: {
    fontSize: 16,
    fontWeight: '700',
    color: '2d6a4f',
    marginBottom: 12,
  },
  infoRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingVertical: 8,
    borderBottomWidth: 1,
    borderBottomColor: 'f0f0f0',
  },
  infoLabel: { fontSize: 14, color: '666' },
  infoValue: { fontSize: 14, fontWeight: '500', color: '1a1a2e', maxWidth: '60%' },
  logoutButton: {
    backgroundColor: 'fff',
    borderWidth: 1,
    borderColor: 'e63946',
    borderRadius: 12,
    paddingVertical: 14,
    alignItems: 'center',
    marginTop: 8,
  },
  logoutText: { color: 'e63946', fontSize: 16, fontWeight: '600' },
});

```

7.6. Login

login.js

Archivo: *app/auth/login.js*

```

// app/auth/login.js
// =====
// Pantalla de Login — Firebase Auth
// Sesión 11: Autenticación con email/password
// =====

import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, KeyboardAvoidingView, Platform,
  ActivityIndicator } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';

```

```

export default function LoginScreen() {
  const router = useRouter();
  const { login, loading, error } = useAuth();
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [localError, setLocalError] = useState('');

  const handleLogin = async () => {
    setLocalError('');
    if (!email.trim() || !password.trim()) {
      setLocalError('Completa todos los campos');
      return;
    }
    try {
      await login(email.trim(), password);
      router.replace('/(tabs)/home');
    } catch (err) {
      setLocalError(err.message);
    }
  };

  return (
    <KeyboardAvoidingView
      style={styles.container}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    >
      <View style={styles.content}>
        <Text style={styles.logo}><img alt="NaturApp logo" data-bbox="378 438 398 450"/></Text>
        <Text style={styles.title}>NaturApp</Text>
        <Text style={styles.subtitle}>Productos naturales a tu alcance</Text>

        {(localError || error) && (
          <View style={styles.errorBox}>
            <Text style={styles.errorText}>{localError || error}</Text>
          </View>
        )}

        <TextInput
          style={styles.input}
          placeholder="Email"
          value={email}
          onChangeText={setEmail}
          keyboardType="email-address"
          autoCapitalize="none"
        />
        <TextInput
          style={styles.input}
          placeholder="Contraseña"
          value={password}
          onChangeText={setPassword}
          secureTextEntry
        />

        <TouchableOpacity style={styles.button} onPress={handleLogin} disabled={loading}>
          {loading ? (
            <ActivityIndicator color="#fff" />
          ) : (
            <Text style={styles.buttonText}>Iniciar Sesión</Text>
          )}
        </TouchableOpacity>

        <TouchableOpacity onPress={() => router.push('/auth/register')}>

```

```

        <Text style={styles.link}>¿No tienes cuenta? <Text
style={styles.linkBold}>Regístrate</Text></Text>
        </TouchableOpacity>

        <TouchableOpacity onPress={() => router.replace('/(tabs)/home')} style={{ marginTop: 12 }}>
        <Text style={styles.skipText}>Continuar sin cuenta →</Text>
        </TouchableOpacity>
    </View>
    </KeyboardAvoidingView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  content: {
    flex: 1,
    justifyContent: 'center',
    padding: 24,
  },
  logo: { fontSize: 60, textAlign: 'center', marginBottom: 8 },
  title: { fontSize: 28, fontWeight: '700', color: '#2d6a4f', textAlign: 'center' },
  subtitle: { fontSize: 14, color: '#888', textAlign: 'center', marginBottom: 30 },
  errorBox: {
    backgroundColor: '#fdecea',
    borderRadius: 8,
    padding: 10,
    marginBottom: 16,
  },
  errorText: { color: '#e63946', textAlign: 'center', fontSize: 13 },
  input: {
    backgroundColor: '#fff',
    borderRadius: 12,
    paddingHorizontal: 16,
    paddingVertical: 14,
    fontSize: 15,
    marginBottom: 12,
    elevation: 1,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.05,
    shadowRadius: 2,
  },
  button: {
    backgroundColor: '#2d6a4f',
    borderRadius: 12,
    paddingVertical: 14,
    alignItems: 'center',
    marginTop: 8,
    marginBottom: 16,
  },
  buttonText: { color: '#fff', fontSize: 16, fontWeight: '700' },
  link: { textAlign: 'center', color: '#666', fontSize: 14 },
  linkBold: { color: '#2d6a4f', fontWeight: '600' },
  skipText: { textAlign: 'center', color: '#999', fontSize: 13 },
});

```

7.7. Register

register.js

Archivo: `app/auth/register.js`


```
// app/auth/register.js
// =====
// Pantalla de Registro – Firebase Auth
// Sesión 11: Registro con createUserWithEmailAndPassword
// =====

import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, KeyboardAvoidingView, Platform,
ActivityIndicator, ScrollView } from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../../src/hooks/useAuth';

export default function RegisterScreen() {
  const router = useRouter();
  const { register, loading, error } = useAuth();
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [phone, setPhone] = useState('');
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [localError, setLocalError] = useState('');

  const handleRegister = async () => {
    setLocalError('');
    if (!name.trim() || !email.trim() || !password) {
      setLocalError('Nombre, email y contraseña son obligatorios');
      return;
    }
    if (password.length < 6) {
      setLocalError('La contraseña debe tener al menos 6 caracteres');
      return;
    }
    if (password !== confirmPassword) {
      setLocalError('Las contraseñas no coinciden');
      return;
    }
    try {
      await register({ name: name.trim(), email: email.trim(), phone: phone.trim(), password });
      router.replace('/(tabs)/home');
    } catch (err) {
      setLocalError(err.message);
    }
  };

  return (
    <KeyboardAvoidingView
      style={styles.container}
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
    >
      <ScrollView contentContainerStyle={styles.content}>
        <Text style={styles.logo}><img alt="NaturApp logo" data-bbox="378 731 398 744"/></Text>
        <Text style={styles.title}>Crear Cuenta</Text>
        <Text style={styles.subtitle}>Únete a NaturApp</Text>

        {(localError || error) && (
          <View style={styles.errorBox}>
            <Text style={styles.errorText}>{localError || error}</Text>
          </View>
        )}

        <TextInput
          style={styles.input}
          placeholder="Nombre completo"

```

```

        value={name}
        onChangeText={setName}
      />
      <TextInput
        style={styles.input}
        placeholder="Email"
        value={email}
        onChangeText={setEmail}
        keyboardType="email-address"
        autoCapitalize="none"
      />
      <TextInput
        style={styles.input}
        placeholder="Teléfono (opcional)"
        value={phone}
        onChangeText={setPhone}
        keyboardType="phone-pad"
      />
      <TextInput
        style={styles.input}
        placeholder="Contraseña (mín. 6 caracteres)"
        value={password}
        onChangeText={setPassword}
        secureTextEntry
      />
      <TextInput
        style={styles.input}
        placeholder="Confirmar contraseña"
        value={confirmPassword}
        onChangeText={setConfirmPassword}
        secureTextEntry
      />

      <TouchableOpacity style={styles.button} onPress={handleRegister} disabled={loading}>
        {loading ? (
          <ActivityIndicator color="#fff" />
        ) : (
          <Text style={styles.buttonText}>Registrarme</Text>
        )}
      </TouchableOpacity>

      <TouchableOpacity onPress={() => router.back()}>
        <Text style={styles.link}>¿Ya tienes cuenta? <Text style={styles.linkBold}>Inicia
sesión</Text></Text>
      </TouchableOpacity>
    </ScrollView>
  </KeyboardAvoidingView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  content: {
    flexGrow: 1,
    justifyContent: 'center',
    padding: 24,
  },
  logo: { fontSize: 50, textAlign: 'center', marginBottom: 8 },
  title: { fontSize: 24, fontWeight: '700', color: '#2d6a4f', textAlign: 'center' },
  subtitle: { fontSize: 14, color: '#888', textAlign: 'center', marginBottom: 24 },
  errorBox: {
    backgroundColor: '#fdecea',
    borderRadius: 8,

```

```

padding: 10,
marginBottom: 16,
},
errorText: { color: '#e63946', textAlign: 'center', fontSize: 13 },
input: {
  backgroundColor: '#fff',
  borderRadius: 12,
  paddingHorizontal: 16,
  paddingVertical: 14,
  fontSize: 15,
  marginBottom: 12,
  elevation: 1,
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 1 },
  shadowOpacity: 0.05,
  shadowRadius: 2,
},
button: {
  backgroundColor: '#2d6a4f',
  borderRadius: 12,
  paddingVertical: 14,
  alignItems: 'center',
  marginTop: 8,
  marginBottom: 16,
},
buttonText: { color: '#fff', fontSize: 16, fontWeight: '700' },
link: { textAlign: 'center', color: '#666', fontSize: 14 },
linkBold: { color: '#2d6a4f', fontWeight: '600' },
});

```

7.8. Detalle de Producto

product/[id].js

Archivo: app/product/[id].js

```

// app/product/[id].js
// =====
// Pantalla de Detalle de Producto
// Sesión 11: Vista detallada con Firestore
// =====

import React, { useEffect, useState } from 'react';
import { View, Text, Image, ScrollView, TouchableOpacity, StyleSheet, ActivityIndicator, Alert } from
'react-native';
import { useLocalSearchParams, useRouter } from 'expo-router';
import { useProducts } from '../src/hooks/useProducts';
import { useCart } from '../src/hooks/useCart';
import { useAuth } from '../src/hooks/useAuth';

export default function ProductDetailScreen() {
  const { id } = useLocalSearchParams();
  const router = useRouter();
  const { user } = useAuth();
  const { getProductById } = useProducts();
  const { addItem } = useCart(user?.id);
  const [product, setProduct] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const load = async () => {
      const p = await getProductById(id);

```

```

        setProduct(p);
        setLoading(false);
    };
    load();
}, [id, getProductById]);

const handleAddToCart = async () => {
    if (!user) {
        router.push('/auth/login');
        return;
    }
    await addItem(product);
    Alert.alert('Agregado', `${product.name} se agregó al carrito`);
};

if (loading) {
    return (
        <View style={styles.center}>
            <ActivityIndicator size="large" color="#2d6a4f" />
        </View>
    );
}

if (!product) {
    return (
        <View style={styles.center}>
            <Text style={styles.errorText}>Producto no encontrado</Text>
        </View>
    );
}

return (
    <ScrollView style={styles.container}>
        <Image
            source={{ uri: product.image || 'https://via.placeholder.com/400x300.png?text=Producto' }}
            style={styles.image}
            resizeMode="cover"
        />
        <View style={styles.content}>
            <View style={styles.row}>
                <Text style={styles.category}>{product.category || 'General'}</Text>
                <Text style={[[styles.stockBadge, product.stock <= 0 && styles.outOfStock]]}
                    {product.stock > 0 ? `Stock: ${product.stock}` : 'Agotado'}
                </Text>
            </View>

            <Text style={styles.name}>{product.name}</Text>
            <Text style={styles.price}>$ / {(product.price || 0).toFixed(2)}</Text>

            <Text style={styles.descTitle}>Descripción</Text>
            <Text style={styles.description}>{product.description || 'Sin descripción disponible.'}</Text>

            {product.benefits && (
                <>
                    <Text style={styles.descTitle}>Beneficios</Text>
                    <Text style={styles.description}>{product.benefits}</Text>
                </>
            )}

            {product.stock > 0 && (
                <TouchableOpacity style={styles.addButton} onPress={handleAddToCart}>
                    <Text style={styles.addButtonText}>🛒 Agregar al Carrito</Text>
                </TouchableOpacity>
            )}
        </View>
    </ScrollView>
);

```

```

    })
  </View>
</ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  image: { width: '100%', height: 280, backgroundColor: '#f0f0f0' },
  content: { padding: 20 },
  row: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 8,
  },
  category: {
    fontSize: 13,
    color: '#2d6a4f',
    backgroundColor: '#e8f5e9',
    paddingHorizontal: 12,
    paddingVertical: 4,
    borderRadius: 12,
    fontWeight: '500',
  },
  stockBadge: { fontSize: 12, color: '#666' },
  outOfStock: { color: '#e63946', fontWeight: '600' },
  name: { fontSize: 22, fontWeight: '700', color: '#1a1a2e', marginBottom: 6 },
  price: { fontSize: 24, fontWeight: '700', color: '#2d6a4f', marginBottom: 16 },
  descTitle: { fontSize: 16, fontWeight: '600', color: '#1a1a2e', marginBottom: 6, marginTop: 10 },
  description: { fontSize: 14, color: '#555', lineHeight: 22 },
  addButton: {
    backgroundColor: '#2d6a4f',
    borderRadius: 12,
    paddingVertical: 16,
    alignItems: 'center',
    marginTop: 24,
  },
  addButtonText: { color: '#fff', fontSize: 17, fontWeight: '700' },
  errorText: { fontSize: 16, color: '#888' },
});

```

7.9. Checkout

checkout.js

Archivo: `app/checkout.js`

```

// app/checkout.js
// =====
// Pantalla de Checkout
// Sesión 11: Confirmar pedido → Firestore
// =====

import React, { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, StyleSheet, ScrollView, Alert, ActivityIndicator }
from 'react-native';
import { useRouter } from 'expo-router';
import { useAuth } from '../src/hooks/useAuth';
import { useCart } from '../src/hooks/useCart';
import { useOrders } from '../src/hooks/useOrders';

```

```

export default function CheckoutScreen() {
  const router = useRouter();
  const { user } = useAuth();
  const { items, total, clearCart } = useCart(user?.id);
  const { createOrder } = useOrders(user?.id);
  const [address, setAddress] = useState('');
  const [notes, setNotes] = useState('');
  const [submitting, setSubmitting] = useState(false);

  const handleSubmit = async () => {
    if (!address.trim()) {
      Alert.alert('Error', 'Ingresa una dirección de entrega');
      return;
    }
    if (items.length === 0) {
      Alert.alert('Error', 'Tu carrito está vacío');
      return;
    }

    setSubmitting(true);
    try {
      const order = await createOrder({
        items: items.map((i) => ({
          productId: i.productId || i.id,
          name: i.name,
          price: i.price,
          quantity: i.quantity,
          image: i.image,
        })),
        total,
        address: address.trim(),
        notes: notes.trim(),
        status: 'pending',
      });

      if (order) {
        await clearCart();
        Alert.alert(
          'Pedido Confirmado',
          'Tu pedido ha sido registrado en Firebase exitosamente.',
          [{ text: 'Ver Pedidos', onPress: () => router.replace('/(tabs)/orders') }]
        );
      }
    } catch (err) {
      Alert.alert('Error', 'No se pudo crear el pedido');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <ScrollView style={styles.container} contentContainerStyle={styles.content}>
      <View style={styles.section}>
        <Text style={styles.sectionTitle}>Resumen del Pedido</Text>
        {items.map((item) => (
          <View key={item.id} style={styles.itemRow}>
            <Text style={styles.itemName}>{item.name} x{item.quantity}</Text>
            <Text style={styles.itemPrice}>$/ {(item.price * item.quantity).toFixed(2)}</Text>
          </View>
        ))}
        <View style={[styles.itemRow, styles.totalRow]}>
          <Text style={styles.totalLabel}>Total</Text>

```

```

        <Text style={styles.totalValue}>$/ {total.toFixed(2)}</Text>
      </View>
    </View>

    <View style={styles.section}>
      <Text style={styles.sectionTitle}>Datos de Entrega</Text>
      <TextInput
        style={styles.input}
        placeholder="Dirección de entrega"
        value={address}
        onChangeText={setAddress}
        multiline
      />
      <TextInput
        style={[styles.input, { height: 80 }]}
        placeholder="Notas adicionales (opcional)"
        value={notes}
        onChangeText={setNotes}
        multiline
      />
    </View>

    <View style={styles.section}>
      <Text style={styles.sectionTitle}>Cliente</Text>
      <Text style={styles.clientInfo}>{user?.name} – {user?.email}</Text>
    </View>

    <TouchableOpacity style={styles.submitButton} onPress={handleSubmit} disabled={submitting}>
      {submitting ? (
        <ActivityIndicator color="#fff" />
      ) : (
        <Text style={styles.submitText}>Confirmar Pedido</Text>
      )}
    </TouchableOpacity>
  </ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f5f5f5' },
  content: { padding: 16 },
  section: {
    backgroundColor: '#fff',
    borderRadius: 12,
    padding: 16,
    marginBottom: 16,
    elevation: 1,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.05,
    shadowRadius: 2,
  },
  sectionTitle: { fontSize: 16, fontWeight: '700', color: '#2d6a4f', marginBottom: 12 },
  itemRow: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingVertical: 6,
  },
  itemName: { fontSize: 14, color: '#333', flex: 1 },
  itemPrice: { fontSize: 14, fontWeight: '600', color: '#333' },
  totalRow: {
    borderTopWidth: 1,
    borderTopColor: '#eee',

```

```
    marginTop: 8,
    paddingTop: 10,
  },
  totallabel: { fontSize: 16, fontWeight: '700', color: '#1a1a2e' },
  totalValue: { fontSize: 18, fontWeight: '700', color: '#2d6a4f' },
  input: {
    backgroundColor: '#f8f8f8',
    borderRadius: 10,
    paddingHorizontal: 14,
    paddingVertical: 12,
    fontSize: 14,
    marginBottom: 10,
    borderWidth: 1,
    borderColor: '#eee',
  },
  },
  clientInfo: { fontSize: 14, color: '#555' },
  submitButton: {
    backgroundColor: '#2d6a4f',
    borderRadius: 12,
    paddingVertical: 16,
    alignItems: 'center',
    marginBottom: 30,
  },
  },
  submitText: { color: '#fff', fontSize: 17, fontWeight: '700' },
});
```


8. Configuración del Proyecto

8.1. package.json

package.json

Archivo: *package.json*

```
{
  "name": "naturapp-firebase",
  "version": "2.0.0",
  "main": "expo-router/entry",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "seed": "node scripts/seedFirestore.js"
  },
  "dependencies": {
    "expo": "~52.0.0",
    "expo-router": "~4.0.0",
    "expo-status-bar": "~2.0.0",
    "expo-linking": "~7.0.0",
    "expo-constants": "~17.0.0",
    "expo-image-picker": "~16.0.0",
    "react": "18.3.1",
    "react-native": "0.76.0",
    "react-native-screens": "~4.0.0",
    "react-native-safe-area-context": "4.12.0",
    "@react-native-async-storage/async-storage": "1.23.1",
    "@expo/vector-icons": "^14.0.0",
    "react-native-gesture-handler": "~2.20.0",
    "react-native-reanimated": "~3.16.0",
    "firebase": "^11.0.0"
  },
  "devDependencies": {
    "@babel/core": "^7.25.0"
  },
  "private": true
}
```

8.2. app.json

app.json

Archivo: *app.json*

```
{
  "expo": {
    "name": "NaturApp",
    "slug": "naturapp-firebase",
    "version": "2.0.0",
    "orientation": "portrait",
    "scheme": "naturapp",
    "userInterfaceStyle": "light",
    "newArchEnabled": true,
    "splash": {
      "backgroundColor": "#1A5276",
      "resizeMode": "contain"
    },
    "ios": {
```

```

    "supportsTablet": true,
    "bundleIdentifier": "com.naturapp.mobile"
  },
  "android": {
    "adaptiveIcon": {
      "backgroundColor": "#1A5276"
    },
    "package": "com.naturapp.mobile"
  },
  "plugins": [
    "expo-router",
    "expo-image-picker"
  ]
}

```

8.3. Navegación — Layouts

Root Layout

Archivo: `app/_layout.js`

```

// app/_layout.js
// =====
// Layout Principal – Expo Router
// Sesión 11: Navegación raíz con Firebase
// =====

import { Stack } from 'expo-router';

export default function RootLayout() {
  return (
    <Stack screenOptions={{ headerShown: false }}>
      <Stack.Screen name="(tabs)" />
      <Stack.Screen
        name="product/[id]"
        options={{ headerShown: true, title: 'Detalle', headerTintColor: '#2d6a4f' }}
      />
      <Stack.Screen
        name="checkout"
        options={{ headerShown: true, title: 'Confirmar Pedido', headerTintColor: '#2d6a4f' }}
      />
      <Stack.Screen
        name="auth/login"
        options={{ headerShown: false }}
      />
      <Stack.Screen
        name="auth/register"
        options={{ headerShown: false }}
      />
    </Stack>
  );
}

```

Tabs Layout

Archivo: `app/(tabs)/_layout.js`

```

// app/(tabs)/_layout.js
// =====
// Layout de Tabs – 5 pestañas

```

```
// Sesión 11: Navegación inferior con iconos
// =====

import { Tabs } from 'expo-router';
import { Text } from 'react-native';

function TabIcon({ name, color }) {
  const icons = { home: '🏠', search: '🔍', cart: '🛒', orders: '📦', profile: '👤' };
  return <Text style={{ fontSize: 22 }}>{icons[name] || '📱'}</Text>;
}

export default function TabsLayout() {
  return (
    <Tabs
      screenOptions={{
        headerStyle: { backgroundColor: '#2d6a4f' },
        headerTintColor: '#fff',
        tabBarActiveTintColor: '#2d6a4f',
        tabBarInactiveTintColor: '#999',
        tabBarStyle: { height: 60, paddingBottom: 8, paddingTop: 4 },
      }}
    >
      <Tabs.Screen
        name="home"
        options={{
          title: 'Inicio',
          headerTitle: '🌿 NaturApp',
          tabBarIcon: ({ color }) => <TabIcon name="home" color={color} />,
        }}
      />
      <Tabs.Screen
        name="search"
        options={{
          title: 'Buscar',
          tabBarIcon: ({ color }) => <TabIcon name="search" color={color} />,
        }}
      />
      <Tabs.Screen
        name="cart"
        options={{
          title: 'Carrito',
          tabBarIcon: ({ color }) => <TabIcon name="cart" color={color} />,
        }}
      />
      <Tabs.Screen
        name="orders"
        options={{
          title: 'Pedidos',
          tabBarIcon: ({ color }) => <TabIcon name="orders" color={color} />,
        }}
      />
      <Tabs.Screen
        name="profile"
        options={{
          title: 'Perfil',
          tabBarIcon: ({ color }) => <TabIcon name="profile" color={color} />,
        }}
      />
    </Tabs>
  );
}
```


9. Script de Poblamiento de Datos

El script `seedFirestore.js` permite poblar Cloud Firestore con datos iniciales de 5 categorías y 10 productos. Se ejecuta con Node.js y requiere las credenciales de Firebase configuradas.

seedFirestore.js

Archivo: `scripts/seedFirestore.js`

```
// scripts/seedFirestore.js
// =====
// Script para poblar Firestore con datos iniciales
// Ejecutar: node scripts/seedFirestore.js
// Requiere: npm install firebase
// =====

const { initializeApp } = require('firebase/app');
const { getFirestore, collection, addDoc, getDocs, deleteDoc, doc } = require('firebase/firestore');

// — CONFIGURACIÓN —
// Reemplazar con las credenciales de tu proyecto Firebase
const firebaseConfig = {
  apiKey: 'TU_API_KEY_AQUI',
  authDomain: 'tu-proyecto.firebaseio.com',
  projectId: 'tu-proyecto-id',
  storageBucket: 'tu-proyecto.appspot.com',
  messagingSenderId: '123456789',
  appId: '1:123456789:web:abcdef123456',
};

const app = initializeApp(firebaseConfig);
const db = getFirestore(app);

// — DATOS DE CATEGORÍAS —
const categories = [
  { id: 'herbal', name: 'Hierbas', icon: '🌿', description: 'Hierbas medicinales y aromáticas' },
  { id: 'oils', name: 'Aceites', icon: '🌱', description: 'Aceites esenciales y naturales' },
  { id: 'teas', name: 'Infusiones', icon: '🍵', description: 'Tés e infusiones naturales' },
  { id: 'supplements', name: 'Suplementos', icon: '💊', description: 'Suplementos naturales' },
  { id: 'superfoods', name: 'Superalimentos', icon: '🥗', description: 'Superalimentos nutritivos' },
];

// — DATOS DE PRODUCTOS —
const products = [
  {
    name: 'Manzanilla Orgánica',
    description: 'Manzanilla orgánica cultivada en los Andes peruanos. Ideal para infusiones relajantes y digestivas.',
    price: 12.50,
    category: 'herbal',
    stock: 50,
    image: 'https://images.unsplash.com/photo-1587593810167-a84920ea0781?w=400',
    benefits: 'Propiedades relajantes, mejora la digestión, antiinflamatoria natural.',
    active: true,
  },
  {
    name: 'Aceite de Eucalipto',
    description: 'Aceite esencial de eucalipto 100% puro. Aroma refrescante con propiedades descongestionantes.',
    price: 28.00,
    category: 'oils',
    stock: 35,
  }
];
```

```

    image: 'https://images.unsplash.com/photo-1608571423902-eed4a5ad8108?w=400',
    benefits: 'Descongestionante, antiséptico, alivia dolores musculares.',
    active: true,
  },
  {
    name: 'Té Verde Matcha',
    description: 'Matcha premium de grado ceremonial. Energía sostenida y alto contenido de
antioxidantes.',
    price: 45.00,
    category: 'teas',
    stock: 25,
    image: 'https://images.unsplash.com/photo-1515823064-d6e0c04616a7?w=400',
    benefits: 'Antioxidante, mejora el enfoque, acelera el metabolismo.',
    active: true,
  },
  {
    name: 'Espirulina en Polvo',
    description: 'Espirulina orgánica en polvo. Superalimento rico en proteínas, vitaminas y
minerales.',
    price: 38.50,
    category: 'supplements',
    stock: 40,
    image: 'https://images.unsplash.com/photo-1622467827417-bbe6e3b18018?w=400',
    benefits: 'Alta en proteínas, desintoxicante, fortalece el sistema inmune.',
    active: true,
  },
  {
    name: 'Quinua Real Orgánica',
    description: 'Quinua real boliviana orgánica. Grano ancestral con proteína completa.',
    price: 22.00,
    category: 'superfoods',
    stock: 60,
    image: 'https://images.unsplash.com/photo-1586201375761-83865001e31c?w=400',
    benefits: 'Proteína completa, rico en fibra, libre de gluten.',
    active: true,
  },
  {
    name: 'Maca Negra en Cápsulas',
    description: 'Cápsulas de maca negra premium de Junín. Energía y vitalidad natural.',
    price: 55.00,
    category: 'supplements',
    stock: 30,
    image: 'https://images.unsplash.com/photo-1556228578-8c89e6adf883?w=400',
    benefits: 'Aumenta la energía, mejora la resistencia, equilibrio hormonal.',
    active: true,
  },
  {
    name: 'Aceite de Coco Virgen',
    description: 'Aceite de coco virgen extra prensado en frío. Uso culinario y cosmético.',
    price: 32.00,
    category: 'oils',
    stock: 45,
    image: 'https://images.unsplash.com/photo-1526947425960-945c6e72858f?w=400',
    benefits: 'Hidratante natural, ácidos grasos saludables, antimicrobiano.',
    active: true,
  },
  {
    name: 'Muña Silvestre',
    description: 'Muña silvestre de las alturas del Perú. Hierba andina con propiedades digestivas.',
    price: 10.00,
    category: 'herbal',
    stock: 55,
    image: 'https://images.unsplash.com/photo-1530968033775-2c92736b131e?w=400',

```

```

    benefits: 'Digestiva, carminativa, aroma mentolado natural.',
    active: true,
  },
  {
    name: 'Infusión de Uña de Gato',
    description: 'Uña de Gato de la selva amazónica. Hierba tradicional para el sistema inmune.',
    price: 18.00,
    category: 'teas',
    stock: 38,
    image: 'https://images.unsplash.com/photo-1564890369478-c89ca6d9cde9?w=400',
    benefits: 'Fortalece el sistema inmune, antiinflamatorio, antioxidante.',
    active: true,
  },
  {
    name: 'Semillas de Chía Orgánica',
    description: 'Semillas de chía orgánica. Ricas en Omega-3, fibra y proteínas vegetales.',
    price: 15.50,
    category: 'superfoods',
    stock: 70,
    image: 'https://images.unsplash.com/photo-1514995669114-6081e934b693?w=400',
    benefits: 'Rico en Omega-3, alto en fibra, fuente de proteína vegetal.',
    active: true,
  },
];

// — FUNCIONES DE SEED —
async function clearCollection(collectionName) {
  const snapshot = await getDocs(collection(db, collectionName));
  const deletes = snapshot.docs.map((d) => deleteDoc(doc(db, collectionName, d.id)));
  await Promise.all(deletes);
  console.log(` Colección '${collectionName}' limpiada (${snapshot.size} documentos)`);
}

async function seedCategories() {
  console.log('\nSembrando categorías...');
  for (const cat of categories) {
    const docRef = await addDoc(collection(db, 'categories'), cat);
    console.log(`  + ${cat.name} (${docRef.id})`);
  }
}

async function seedProducts() {
  console.log('\nSembrando productos...');
  for (const prod of products) {
    const docRef = await addDoc(collection(db, 'products'), {
      ...prod,
      createdAt: new Date().toISOString(),
    });
    console.log(`  + ${prod.name} (${docRef.id})`);
  }
}

async function main() {
  console.log('=== NaturApp Firestore Seed ===');
  console.log('Proyecto:', firebaseConfig.projectId);

  try {
    // Limpiar colecciones existentes
    console.log('\nLimpiando colecciones...');
    await clearCollection('categories');
    await clearCollection('products');

    // Sembrar datos

```

```
    await seedCategories();
    await seedProducts();

    console.log(`\n✅ Seed completado exitosamente`);
    console.log(`    ${categories.length} categorías`);
    console.log(`    ${products.length} productos`);
  } catch (error) {
    console.error(`\n❌ Error:`, error.message);
    if (error.message.includes('api-key-not-valid') || error.message.includes('configuration')) {
      console.log(`\n⚠️ Debes configurar las credenciales de Firebase en este archivo.`);
      console.log(`    Ir a https://console.firebase.google.com/ y copiar tu config.`);
    }
  }
  process.exit(0);
}

main();
```


10. Guía de Instalación y Ejecución

10.1. Requisitos Previos

Node.js 18+, npm 9+, Expo CLI, y una cuenta de Firebase con un proyecto creado.

10.2. Configurar Firebase

1. Ir a <https://console.firebase.google.com/> y crear un nuevo proyecto.
2. Agregar una aplicación web (icono `</>`) y copiar la configuración.
3. Habilitar Authentication con el proveedor Email/Password.
4. Crear una base de datos Cloud Firestore en modo de prueba.
5. Habilitar Cloud Storage.
6. Reemplazar los valores placeholder en `src/services/firebaseConfig.js` con tus credenciales reales.

10.3. Instalar y Ejecutar

```
# Descomprimir el proyecto
unzip NaturApp_S11_Firebase_Proyecto.zip
cd NaturApp_S11_Firebase

# Instalar dependencias
npm install

# (Opcional) Poblar Firestore con datos iniciales
node scripts/seedFirestore.js

# Iniciar la aplicación
npx expo start
```

10.4. Modo sin Firebase (Fallback)

La aplicación funciona sin configurar Firebase gracias al sistema de fallback integrado. Se cargan 10 productos y 5 categorías desde datos locales. La autenticación crea un usuario demo local. Esta característica permite probar la interfaz inmediatamente sin necesidad de configurar credenciales.

11. Modelo de Datos en Cloud Firestore

11.1. Colecciones

Colección	Campos Principales	Tipo
products	name, description, price, category, stock, image, benefits, active	Documento
categories	id, name, icon, description	Documento
orders	userId, items[], total, status, address, notes, createdAt	Documento
users	name, email, phone, role, createdAt	Documento
users/{uid}/cart	productId, name, price, quantity, image	Subcolección

11.2. Reglas de Seguridad Sugeridas

Se recomienda configurar reglas de seguridad en Firestore para que cada usuario solo pueda leer/escribir su propio carrito y sus propios pedidos, mientras que los productos y categorías sean de lectura pública.

12. Conclusiones

La integración de Firebase en NaturApp demuestra cómo una aplicación móvil puede aprovechar servicios Backend-as-a-Service para simplificar la infraestructura sin sacrificar funcionalidad. Los principales logros de esta implementación son:

Autenticación robusta con Firebase Auth que soporta persistencia de sesión, manejo de errores detallado y un sistema de fallback para desarrollo. Cloud Firestore proporciona una base de datos NoSQL escalable con sincronización en tiempo real, mientras que Firebase Storage maneja el almacenamiento de imágenes de forma eficiente.

El patrón MVVM se mantiene limpio gracias a la separación entre servicios (firestoreService, storageService), ViewModels (Custom Hooks) y Views (pantallas con Expo Router). El sistema de fallback garantiza que la aplicación es completamente funcional para desarrollo y pruebas sin necesidad de configurar un proyecto Firebase real.

Esta arquitectura es escalable, mantenible y sigue las mejores prácticas recomendadas tanto por Firebase como por la comunidad de React Native.